

УПРАВЛЕНИЕ API / ДАННЫЕ / БАЗЫ ДАННЫХ

GraphQL против OpenAPI: плюсы и минусы для управления данными

Технический директор компании из списка Fortune 50 оценивает плюсы и минусы стандартов OpenAPI и GraphQL API, а также их значимость для управления данными.

12 августа 2024 года, 12:55 [Кен Стотт](#)



Изображение предоставлено Джей Ди Хэнкоком, CC BY 2.0 (через [Flickr](#))

[Hasura](#) спонсировала этот пост.

Кока-кола против Пепси ... Али против Фрейзера ... «Звездные войны» против «Звездного пути» ... Мас против ПК ... GraphQL против OpenAPI ...

Ладно, я просто сгущаю краски. Не думаю, что противостояние GraphQL и OpenAPI стоит в одном ряду с другими фанатскими движениями. Но в сфере

API играют важнейшую роль в потреблении данных и, по сути, являются необходимым условием эффективного управления данными. Об этом говорится во множестве статей об архитектурах **data mesh**. Эффективное управление данными и API в сочетании со сбором и использованием метаданных, создаваемых API, имеют решающее значение для получения информации о потреблении, создания циклов обратной связи и разработки самокорректирующихся процессов.

Итак, какова **стратегия использования API** в вашей организации? Как эти решения влияют на цели управления данными? Наличие целевого состояния и осознанный подход окупаются. Если вы отвечаете за управление данными, но не за технологии, вам все равно нужно участвовать в управлении API и дать понять, что вы заинтересованы в архитектуре. Если вы отвечаете за технологии и поддерживаете деятельность по управлению данными, вы должны выступить за **решения, которые укрепляют управление данными**.

OpenAPI против GraphQL: сходства и различия

Несмотря на то, что существуют и другие стандарты API, такие как REST, gRPC, SOAP и JSON-RPC (или, если хотите, Sierra Mists, Battlestar Galactica или Linux), я сосредоточусь на GraphQL и OpenAPI как на конкретных примерах, но вы можете применить эти идеи к своему любимому стандарту API.



Hasura упрощает доступ к данным, мгновенно создавая GraphQL API на основе баз данных и сервисов, чтобы команда разработчиков (или потребители API) могла сразу приступить к работе. Сама природа GraphQL и динамический подход Hasura упрощают интеграцию и итерации.

[Узнать больше →](#)

ПОСЛЕДНЕЕ СООБЩЕНИЕ ОТ ХАСУРЫ

Data access layer: Unlocking the full potential of financial data

24 March 2025

Data products, data contracts: A new model for data management in financial services

18 March 2025

GraphQL и OpenAPI определяют соглашения между потребителями и поставщиками данных, но у них есть существенные различия в возможностях и спецификациях контрактов. Какой бы из этих подходов вы ни использовали, важно действовать обдуманно, понимать свой сценарий использования и проектировать систему так, чтобы добиться наилучшего результата.

So now that the basics are out of the way, let's compare GraphQL and OpenAPI and evaluate their relevance to data governance based on five key criteria: unified semantic layer, data access, composability, future direction and extensibility, and then look at which is better for data governance.

TRENDING STORIES

1. [Timescale Hyperfunctions Ease PostgreSQL Query Complexities](#)
2. [Maria DB Gets Reactive with a Non-Blocking Connector for Java](#)
3. [6 Tips for Better Data Science in the Cloud](#)
4. [Distributed SQL Takes Databases to the Next Level](#)
5. [Alation Delivers a New Data Governance App](#)

Unified Semantic Layer

An API must establish and maintain a unified semantic layer (or **data access model**) across multiple data producers. It is a control layer that defines entities and relationships, supports a data catalog, gives consistency to data requests, provides metadata to governance activities, and helps guide and organize proper data usage.

GraphQL

A GraphQL endpoint includes data entities, attributes and relationships defined in platform-agnostic terms. It can be managed as a single artifact or

and unions in its type system.

OpenAPI

OpenAPI uses the JSON Schema standard to define data types and the data validation model for an API endpoint. As with GraphQL, you can manage OpenAPI endpoints in a federated fashion. Each endpoint expresses relationships between other OpenAPI documents through a URL scheme. Historically, maintaining and evolving a large, unified data model using JSON Schema has been challenging.

Data Access

APIs must provide a data access approach aligned with data products. They need tech-facing self-service capabilities, enforce fine-grained (field-level) access controls, and support a variety of client transports and protocols.

GraphQL

GraphQL lets you specify which data elements can be directly **queried** or subscribed to. Since clients request data at the field level, you can implement fine-grained access controls within the semantic layer.

The elements that can be requested (or in GraphQL terms, the elements at the root of the query type) are akin to a “**data product**” and have strong correlation to an underlying data set. In its broadest definition, data products can be anything from a single data set to a 360-degree customer-reporting application. Definitions are a slippery slope, but in my opinion, GraphQL SDL does an excellent job of defining something that looks a lot like a data catalog of **data products**, at least in its narrow definition.

GraphQL also allows you to specify additional predicates for refining the request, such as filters or sorts. It can describe how to delete, update or create additional data (known as mutations), which is required to support operational workloads. Additionally, you can define fragments, which are similar to subroutines, for reusability and composability.

GraphQL’s query language does not incorporate other standards like HTTP or REST. It is agnostic about how the client and server communicate the request and result. A server delivers an HTTP JSON payload from an HTTP POST or persistent sockets (subscriptions). Community solutions provide file format-

OpenAPI

OpenAPI uses **JSON Schema** as its foundation and relies on HTTP REST. While developers use REST verbs (GET, POST, PATCH, etc.) to define the class of an operation; their use is not enforced by OpenAPI tooling. All transactions occur over HTTP, allowing an API to define its inputs through the Uniform Resource Identifier (URI) path, query or HTTP request body.

Developers usually manage required or optional inputs using the JSON Schema standard. In this approach, you define access controls at the Remote Procedure Call (RPC) level. Response customization is proprietary since no such concept exists in the OpenAPI standard. An API can define its payload in any format, so JSON and file-format payloads are possible.

Composability

Composability reduces the burden on data producers and gives additional self-service capabilities to clients. It allows producers to generate data products that are more generalized and one step removed from specific use cases; therefore, they're more reusable. Within the context of data access, a composable system has four key features:

1. **Shape:** Defines the response's structure or shape by specifying its data elements, organization and any relationships between them.
2. **Combine:** Assembles data products (tables, views or APIs) into a cohesive whole. These data products can be reused across different operations.
3. **Reuse:** Emphasizes reusable building blocks by defining concepts (like data products, query fragments or relationships) to create a foundation for flexible composition.
4. **Tailor:** Customizes a data response for a specific use case or logical operation by combining components to achieve a desired outcome.

GraphQL

GraphQL's query language allows the requestor to define desired data elements and traverse defined data relationships. The server handles traversing relationships and returns the request as a single result. This reusability of defined relationships is one way **GraphQL provides composability** — GraphQL's superpower.

OpenAPI

handles data composition, which often leads to over-fetching data and multiple calls between frontend and backend systems to stitch together a single logical request.

Unnecessary data movement incurs costs, including for data governance. Over-fetching data in a governed environment increases complexity and costs for monitoring and security, particularly in regulated environments.

Future Focus

To assess an API standard's viability, ask questions like: What is the probability the standard will align to data-centric use cases moving forward? How does its mission align with data-centric use cases? Are there dependencies or constraints that may complicate its evolution?

GraphQL

GraphQL is a self-contained, coherent, open standard focused on data-centric APIs, supported by an active standards body. The next iteration will focus on issues like federation and streaming to adapt to additional data-centric use cases and improve its alignment with **data mesh architectures**.

OpenAPI

OpenAPI relies on JSON Schema and HTTP REST. Its stated purpose is to allow humans and computers to understand an API without access to source code. This "standards mashup" creates challenges in tooling (perhaps solvable), and its stated purpose is not data-centric but more open-ended. **The next iteration of OpenAPI**, version 4, will focus on AI and generative AI (GenAI).

Extensibility

A well-designed standard should incorporate extensibility in a digital-friendly, maintainable way to address unique organizational needs.

GraphQL

GraphQL is designed to be extensible and has features like custom directives, custom scalars and extendable types. Directives modify the behavior of operations. GraphQL defines standard directives like `@deprecated`, which indicates that a field has a sunset. Additionally, it can define custom directives, e.g., `@source`, which could document a data element's expected

computing.

OpenAPI

OpenAPI has limited extensibility based on conventions. You must add custom metadata to an OpenAPI specification by creating custom fields with an `x-
<tag-name>` type of notation. This convention effectively tells any standard tooling to ignore these fields.

Final Assessment: Alignment to Data Governance

It's never going to be a binary decision. However, establishing standards and providing thoughtful guidance to practitioners will make a meaningful difference in data governance outcomes. Within the limited scope I set, here is my assessment.

GraphQL

GraphQL SDL has a sophisticated type system, a **well-defined query language** and a flexible, structured extensibility approach. As a data-centric API standard, it can accomplish everything that OpenAPI can. Aligning to underlying data sources is more straightforward with metadata, and tooling makes it easier to support data governance requirements.

Due to its specialized focus, GraphQL can offer reference libraries implemented in multiple languages. These libraries parse GraphQL schemas and queries and connect them to code, facilitating the integration of databases and business logic into a response.

This request-handling model provides hooks that support data governance objectives. For example, it can evaluate and record data accuracy inline or track data use with rich contextual information. This well-designed, at-the-edge data-processing model is adaptable and customizable to meet data governance needs.

OpenAPI

OpenAPI is widely used, well-known and beloved by many developers. While you can build a data delivery platform with OpenAPI, it will be more DIY. You will need to make additional design decisions and acquire tooling to make this strategy support data governance objectives.

Conclusion

GraphQL seriously five years ago, but the technology landscape has evolved. Hand-coding GraphQL resolvers is **not on my bucket list**; what makes this feasible is automated GraphQL. There are credible vendors with excellent GraphQL automation capabilities that make this a viable and attractive approach.

OpenAPI, gRPC and API gateway vendors are also practical solutions. They align more with historical practices and perhaps are an easier “sell” to certain stakeholders. But they are less capable out of the box. Establishing internal standards and practices to maintain lineage back to underlying data, generating audit evidence, managing fine-grained access controls and supporting consistent experiences across multiple client protocols requires technical leadership, architecture and design across a distributed data delivery environment, and that’s hard to do.

TNS



Ken was most recently the Director of Architecture for key data initiatives at a Fortune 50 Bank. He crafts data strategies for various initiatives while navigating complex regulatory challenges and competitive pressures. His experience spans risk management consulting at Deloitte,...

[Read more from Ken Stott →](#)

TNS owner Insight Partners is an investor in: OpenAI, Uniform.

SHARE THIS STORY



TRENDING STORIES

1. Timescale Hyperfunctions Ease PostgreSQL Query Complexities
2. Maria DB Gets Reactive with a Non-Blocking Connector for Java
3. 6 Tips for Better Data Science in the Cloud
4. Distributed SQL Takes Databases to the Next Level

INSIGHTS FROM OUR SPONSOR



Hasura makes data access easy, by instantly composing a GraphQL API that is backed by databases and services so that the developer team (or API consumers) get immediately productive. The nature of GraphQL itself and Hasura's dynamic approach makes integration and iteration easy.

[Learn More →](#)

Data access layer: Unlocking the full potential of financial data

24 March 2025

Time-traveling through your data architecture: Using data agents to understand change

19 March 2025

Data products, data contracts: A new model for data management in financial services

18 March 2025

How PromptQL achieves 100% accuracy for AI on enterprise data

11 March 2025

Hasura: Powerful access control on MongoDB data

5 March 2025

Escaping the data doom loop in financial services

4 March 2025

most recent TNS articles in your inbox each day.

svo1975pvn1982@gmail.com

SUBSCRIBE

The New Stack does not sell your information or share it with unaffiliated third parties. By continuing, you agree to our [Terms of Use](#) and [Privacy Policy](#).

ARCHITECTURE

Cloud Native Ecosystem
Containers
Databases
Edge Computing
Infrastructure as Code
Linux
Microservices
Open Source
Networking
Storage

OPERATIONS

AI Operations
CI/CD
Cloud Services
DevOps
Kubernetes

ENGINEERING

AI
AI Engineering
API Management
Backend development
Data
Frontend Development
Large Language Models
Security
Software Development
WebAssembly

CHANNELS

Podcasts
Ebooks
Events
Webinars
Newsletter

THE NEW STACK

[About / Contact](#)
[Sponsors](#)
[Advertise With Us](#)
[Contributions](#)



Community created roadmaps, articles, resources and journeys for developers to help you choose your path and grow in your career.

[Frontend Developer Roadmap](#)
[Backend Developer Roadmap](#)
[Devops Roadmap](#)

FOLLOW TNS



© The New Stack 2026

[Disclosures](#)
[Terms of Use](#)
[Advertising Terms & Conditions](#)
[Privacy Policy](#)
[Cookie Policy](#)